Requiem Registry - Final Document

By: Samantha Cook, Lucas Gamboa, Bricen Hicks

Table of Contents

Table of Contents **2**
Introduction **4**
Technology **5**
Easy or Difficult to Learn 5
What we have changed 5
Vercel and git clashing 5
Next.js to React 5
Neon to Supabase 6
Design **7**
SRC Directory 7
Cemetery Subdirectory 8
AddCemetery.js 8
CemeteryDetails.jsx 8
CemeteryList.jsx 8
CemeteryRow.jsx 8
Components Subdirectory 9
NavBar.js 9
Helper Subdirectory 9
supabaseClient.js 9
Map Subdirectory 9
Map.jsx 9
UserMap.jsx 10
Person Subdirectory 10
AddPerson.js 10
PersonDetails.jsx 10
PersonList.jsx 10
PersonRow.jsx 11
PersonCard.js 11
Plot Subdirectory 11
PlotList.jsx 11
PlotRow.jsx 11
Resources Subdirectory 11
CSS SubSubdirectory 11
PNGs Sub Subdirectory 12
Sign In Subdirectory 12
How to deploy or build the application **13**
To deploy the project locally: 13
To deploy the project via Vercel: 13

Database Connection:..... 13
Known issues.....15
Incomplete Features..... 15
Safe Deletion and Restoration..... 15
Next of Kin System..... 15
Administrative Hierarchy..... 15
Google Maps API Integration..... 15
Mobile Version..... 16
Validation of Birth and Death Date..... 16
Sort List Page Entries By Attribute..... 16
Search List Page Entries by Attribute..... 16
Adding or Replacing a Cemetery’s Map after Creating it..... 16
Transform Position of Plots on Map Page..... 17
Feedback on Add Person Page..... 17
Bugs..... 17
Overlapping Plot Icons on Map Page..... 17
Deleting Plots via Map Page Interface Ineffective..... 17
Assigning a Person to Multiple Plots..... 18
Mapless Cemeteries on the Map Page..... 18

Introduction

Imagine you want to find a loved one's grave, and you know that it is in a small cemetery in a rural area; where do you begin to find their grave? Most small cemeteries in rural areas have little to no management or proper records to help point you in the right direction. This problem and a more appropriately streamlined management system are what the Requiem Registry aims to solve. Requiem Registry provides a user-friendly user interface with which cemetery management personnel can interact. This will allow them to store records in one place, preventing data duplication or redundancy. Requiem Registry also provides a simple, general user side to provide anyone with a quick search to find if a loved one rests in a certain cemetery. The home page offers a selection bar to select which cemetery you would like to query; if the user is unsure of which cemetery to search, there is a select all option that will search in all the cemeteries. Now that you have established your loved one is in said cemetery, the next obstacle is where their plot is located within the cemetery. Our website also provides a map of the cemetery, which will give a visual marker of the plot’s location on a map to help ease finding it within the cemetery.

Technology

Technology used for the project includes React, HTML, CSS, and JavaScript for the front end of the website. The backend is written in JavaScript, and the database is a PostgreSQL database hosted on Supabase. The website is deployed via Vercel from the GitHub repository.
React was chosen as it is a popular and commonly used language, which is a valuable skill to learn. Styling is commonly achieved via CSS, thus makes for customization of the pages.

Easy or Difficult to Learn

React JavaScript SQL

What we have changed

Vercel and git clashing

We originally planned to start with a simple project using Vercel, Next.js, and Neon. This project would be a template that was already deployed, and then we would change it to be our functionality. However, we ran into a problem with this approach because any time someone other than the owner of the Vercel-linked Gmail made a Git commit, Vercel flagged it as unauthorized. This made it impossible for the entire team to collaborate and slowed down our progress. After looking into other options, we discovered React and Supabase. We did some research on both technologies and, as a group, decided to pivot and build the project using this new tech stack instead.

Next.js to React

At first, we planned to use Next.js for our project. We chose it because it's a popular framework and works well with Vercel. However, as we got deeper into the project, we realized that we didn't need many of the features that Next.js offers, such as server-side rendering and file-based routing. Our website didn't require those advanced features, and keeping them in the project only added extra complexity that we didn't need.

Switching to plain React turned out to be a better option. It gave us more control, was easier to understand, and was a better match for our team's skill level. React is also very common and has a huge community, which made it easy to find tutorials, examples, and help when we needed it. Since we were already using client-side rendering, React let us focus more on actually building the website without getting stuck

on extra configuration. Overall, using React made development smoother and saved us time.

Neon to Supabase

At first, we planned to use Neon as our database. We picked it because the Vercel website recommended it, and it seemed like a good fit for modern web apps. However, as we began building our project, we ran into some issues. One of the biggest problems was that setting up the connection between Neon and our app was more difficult than expected. It wasn't very beginner-friendly, and we spent a lot of time trying to get the database to communicate properly with the rest of our project.

That's when we started looking into Supabase, and it quickly became clear that it was a much better choice. It was easier to connect to React, which made development smoother right away. Supabase still uses PostgreSQL like Neon, but it also comes with many helpful tools built in. It offers user login and permissions, so we could easily manage who could view or edit data. The live updates made it easier to see our changes in real time when adding or deleting data. We could also store images and other files directly in Supabase, which was perfect for things like cemetery maps. Plus, the Supabase dashboard made learning and using SQL a lot easier. All of these features saved us time and made development much simpler, so switching to Supabase was an easy decision.

Design

SRC Directory

The design of Requiem Registry is broken up into several different directories to keep the information separated and organized. The src file of the website is divided into the following subdirectories: cemetery, components, helper, map, person, plot, resources, and sign-in. Each of those will be discussed in further detail below; before that, the remaining files within the src directory will be discussed. The src directory file also contains App.jsx, HomePage.jsx, index.js, logo.svg, reportWebVitals.js, and setupTests.js. The files, including logo.svg, reportWebVitals.js, and setupTests.js are predefined when setting up a React project. These files were never messed with, meaning whatever is contained within those files is the prefabricated instructions.
The index.js file contains the GoogleAuth provider and the App.jsx file. This helps with the structure of the website. The next file is the App.jsx, which contains all the screens/pages within the website. The App.jsx takes care of the Google Sign-in and contains a route for each page of the website. The Navibar is also instantiated on the app page. The app page also contains boolean triggers to distinguish what it should allow access to based on whether the user has logged in or is just a general user. The navigation bar will update what navigation options it has based on the login status. The routes also change based on login status, as the map page will have different privileges based on user type, and the admin user will have access to many more features on the navigation bar.
The Homepage.jsx is also in the src directory and contains the user interface for the homepage as well as the logic for the homepage. The HomePage contains a header bar, cemetery dropdown, cemetery details, search bar, and search results. The cemetery details also contain a link to the cemetery information page, as well as the person search results, which have a link to the person detail page. The header of the homepage has the logo, the name of the website, and the cemetery dropdown. This dropdown will edit where the search bar is querying in the database. Selecting all will allow the search bar to search any person in the database, whereas a selection in the cemetery dropdown will limit search queries to only people who are in a plot that is assigned to that selected cemetery. This will make for a quicker search as it will limit the number of search results. Once a cemetery selection is made, the database will be queried for all the information on that cemetery and displayed under the header. The search results from the search bar will be displayed in a list below that, the details on each person result will be brief, but clicking the person's name will direct to the person's detail page, which will display all the data contained in the database on that person. The same goes for the cemetery details; clicking the cemetery name will take the user to the

cemetery detail page and will render all the information contained in the database for that specific cemetery.

Cemetery Subdirectory

The cemetery directory contains four classes, including AddCemetery.js, CemeteryDetails.jsx, CemeteryList.jsx, and CemeteryRow.jsx. These four classes contain all the logic and data processing for the cemeteries.

AddCemetery.js

The add cemetery class handles the form of adding a cemetery, so there is code to display that form. The class also has the logic to process that data and insert the data into the database. The class imports the database client from the helper class and pushes data into it as long as it meets the requirements and the connection is good.

CemeteryDetails.jsx

The cemetery details class has the code to display the cemetery's details on its own page after the user links the name, which is a hyperlink to navigate to this page. The class has logic to distinguish if a user is logged in as an admin, as well as if it was routed to this page from the cemetery list or home page. Those two things are important, as that is what enables or disables the ability to edit or delete the cemetery. This class is connected to the database client and makes fetch requests, update requests, as well as deletes.

CemeteryList.jsx

The cemetery list class contains a list of all the cemeteries in the database. The list provides a simplified view with just the name, address, and email. Each cemetery's name is a hyperlink that routes to that cemetery's detail page as mentioned above. This class just uses a SELECT * query to get all cemeteries from the cemetery table.

CemeteryRow.jsx

The cemetery row is a reusable component, which is what is rendered in the cemetery list as each element in the table. The cemetery row is each entry that is viewed in the list and provides the route to the cemetery detail page.

Components Subdirectory

NavBar.js

The NavBar class contains all the logic and styling for the navigation bar that is on the left side of the website. The navigation bar contains the logic for all the routing of the buttons of the navigation bar. This class styles the home button the be the logo, as well as having styling to make the list option almost like a sub menu. This allows for a dropdown of the three list pages: cemetery, people, and plot. The navigation bar class also contains logic to check if the user is logged in as an admin and, if so, updates accordingly. The navigation bar will only display the homepage and a map page option for a general user. Once an admin is logged in, the navigation bar will update to show the homepage, map, add person, add cemetery, and lists pages.

Helper Subdirectory

The helper directory contains the supabaseClient class.

supabaseClient.js

This class is simply an import statement of a SupabaseClient with the URL and anonymous Key needed to connect to the database. The class defines a Supabase client with that information and exports the instance of the Supabase client, as this is what is used throughout the backend to create, read, update, and delete data from the database.

Map Subdirectory

The map subdirectory contains the map and usermap classes. Both of these will be outlined in more detail below.

Map.jsx

The map page contains a large majority of the logic needed for the website to function. The map class handles the connection of people to plots and plots to cemeteries. The map class fetches all the cemeteries and people. This information is what is used to display the map, place plots, and then assign people to plots. The plots can be reassigned or put back empty, so there is logic for that; the map can add and delete plots as well. The map page also has a sidebar, which displays information about the cemetery; the logic to pull that information from the database is contained in this class as well.

UserMap.jsx

The user map page is basically the same as the map page, just in a read-only state. The user map class contains logic to fetch cemeteries, plots, and people as needed. The map for users does not have any editing abilities; it is just showing what has been updated on the map page for the admin. This allows users to navigate the map, look at plots to see who is in them, and go to the person's detail page from the link shown on occupied plots. The map page also has a sidebar, which displays information about the cemetery; the logic to pull that information from the database is contained in this class as well.

Person Subdirectory

The person directory contains five classes, including AddPerson.js, PeopleList.jsx, PersonCard.js, PersonDetails.jsx, and PersonRow.jsx. These five classes contain all the logic and data processing for the people.

AddPerson.js

The add person class handles the form of adding a person, so there is code to display that form. The class also has the logic to process that data and insert it into the database. The class imports the database client from the helper class and pushes data into it as long as it meets the requirements and the connection is good.

PersonDetails.jsx

The cemetery details class has the code to display the cemetery's details on its own page after the user links the name, which is a hyperlink to navigate to this page. The class has logic to distinguish if a user is logged in as an admin, as well as if it was routed to this page from the cemetery list or home page. Those two things are important, as that is what enables or disables the ability to edit or delete the cemetery. This class is connected to the database client and makes fetch requests, update requests, and deletes.

PersonList.jsx

The person list class contains a list of all the people in the database. The list provides a simplified view with just the full names, birth date, and death date if applicable. Each name is a hyperlink that routes to the person's detail page as mentioned above. This class just uses a SELECT * query to get all people from the person table.

PersonRow.jsx

The person row is a reusable component that is rendered in the person list for each element in the table. The person row is each entry that is viewed in the list and provides the route to the person detail page.

PersonCard.js

The personCard class contains the component that is used to display the person's information on the Home Page, and the information is styled with CSS to look like cards. The personCard class also contains the logic to route to the person detail page from the home screen. There is also logic for choosing how much information to display.

Plot Subdirectory

The person directory contains two classes, including PlotList.jsx and PlotRow.jsx. These two classes contain all the logic and data processing for the people.

PlotList.jsx

The plot list class contains a list of all the plots in the database based on the cemetery. The list provides a view with the plot ID, cemetery ID, latitude, longitude, tenant name, plot number, and a column called actions. The actions column holds the delete button. There is a dropdown menu to select which cemetery you would like to view plots from. The list will only show the plots that have been added to the map. The plot can be empty and still populate the list, but if the plots have not been dropped on the map, they will not appear. Empty plots that have been put on the map will say unassigned under the tenant name in the list until a name has been assigned.

PlotRow.jsx

The plot row class is what renders each entry in the plot list table. Every plot that is added to a cemetery will be an instance of the class, which is like a reusable component. This is very similar to the cemetery row and person row.

Resources Subdirectory

CSS SubSubdirectory

There is some inline CSS, but the vast majority of it is in the resources folder under the CSS subdirectory. The CSS contains a map, forms, person list, person details, and plots

list CSS files. The names of the CSS files are pretty straightforward as to what they contain styling for.

PNGs Sub Subdirectory

The PNG subdirectory just contains the logo files in PNG. There are two different-sized logos, so there are two copies inside.

Sign In Subdirectory

The sign-in class takes care of all the Google Authentication for signing into the website as an administrator user. This class also contains some inline CSS to style the sign-in button, the sign-out button, as well as the size of the pop-up window that has Google sign-in. In the system's current state, the Google authentication will only allow one email; this is something that, on scale should allow more than one email account, but this raises concerns about a hierarchy in the system of different accounts and account permissions, so as it stands only one person has a admin account.

How to deploy or build the application

To deploy the project locally:

To deploy the website on a local machine, the code source needs to be downloaded to your computer via the GitHub repository. The code can be opened in an IDE such as VSCode or WebStorm. The following commands should be run to get the website running locally:

- npm install
- npm start

After running these commands, the website will be running on localhost, which can be accessed in any web browser. The database should be connected, but if the original database connection is not working, there are database connection instructions below on how to create your own database and connect it to the website.

To deploy the project via Vercel:

To deploy the project via Vercel, the project repository needs to be forked from the GitHub repository. Once the project is forked to a GitHub account, that GitHub account can be used to create an account on Vercel. Once an account has been created on Vercel, the user will be able to select a project from their GitHub repositories to deploy, which allows for quick and easy deployment.

Database Connection:

To create a new database connection, below are instructions on how to set up the database and connect it to the website. The database can be created in Supabase using the following SQL commands:

```
-- Table Creation
create table cemetery (
  id serial primary key,
  name text,
  description text,
  address text,
  phone_number text,
  capacity int2,
  email varchar(50),
  image_URL text

);

create table person (
```

```
  id serial primary key,
  f_name text,
  m_name text,
  l_name text,
  suffix text,
  birth_date date,
  death_date date,
  biography text,
);

create table rr_plot (
  plot_id int8,
  ceme_id int references cemetery NOT NULL,
  coord_lat decimal(9,6) NOT NULL,
  coord_long decimal(9,6) NOT NULL,
  tenant_id int references person
  primary key (plot_id, ceme_id)
);
```

These SQL commands should create the necessary tables, and Supabase will provide the needed API keys to put into the project's supabaseClient.js file. There is a tab in Supabase called APIDocs, which will walk you through the necessary steps to get the API keys needed for front-end code connection.
A Supabase bucket will need to be created under the Storage tab called cemetery-images. This will need to be connected to the image_URL field in the cemetery table. This is how the PNGs for cemetery maps will be stored.

Known issues

Incomplete Features

Safe Deletion and Restoration

Deleting any entries in the database is currently permanent. While unintended deletions are partially guarded against with a pop-up that asks for confirmation, more can be done. The solution is to add a boolean attribute called “isDeleted” to all tables that is initially set to false, with it being set to true if it is “deleted” using the administrative side of the website. Entries with the attribute set to true could then be filtered from all query results on the websites, removing them from the site, but not from the database. If desired, these pseudo-deleted entries can be restored by setting the boolean back to false. Whether this process could be done in the website’s interface or not is something to consider later.

Next of Kin System

The person’s table in the database was initially designed so that each entry included attributes to record the name and contact information for a person’s next of kin. This can be used by caretakers to reach out to a loved one of the deceased if a matter they should be informed about came up. Currently, these attributes cannot be read from or written to with the website’s interface. The solution to this would be to include these attributes in the add person form and edit person form, and set them to display on the administrative side of the person information page.

Administrative Hierarchy

Administrators should only have read and write access to the cemeteries to which they have been explicitly given access. Currently, Requiem Registry only has one acting administrator. This person has read and write access to all cemeteries, plots, and people in the database. A solution is to create an administrative hierarchy with clearly defined responsibilities and privileges for each role or level, then implement it by adding one or more tables to the database to track these relationships and serve as a basis for privilege validation.

Google Maps API Integration

Initially, the Map Page was supposed to use Google Maps to take the address of the selected Cemetery and load an overhead view of its coordinates on the page. This

function could not be completed due to time constraints. For this reason, there is no explicit solution to this issue other than to have the time to study the API and integrate it properly.

Mobile Version

While Requiem Registry's website and its functionalities can be accessed with a mobile device, its visual design is the same as the desktop version. This results in several visual bugs that, while not affecting the website's functionality, hamper navigation and presentation. The solution to this issue would be to correctly implement a responsive website layout.

Validation of Birth and Death Date

The birth date and death date of a person are currently not checked for validity, making it possible to add a person who died centuries before they were born. The solution to this is to check that the death date provided is not less than the birth date before inserting or updating an entry to the person table.

Sort List Page Entries By Attribute

Entries in the List Pages are currently sorted by when they were added, with older entries at the top and newer entries at the bottom. This makes finding entries difficult, an issue that will become more prominent as the database grows. The solution is to allow the list to be sorted by a chosen attribute with the option to toggle between ascending and descending order.

Search List Page Entries by Attribute

Specific entries in the List Pages can only be found by scrolling down the list until it is found. This makes finding entries difficult, an issue that will become more prominent as the database grows. The solution is to add a search bar for each attribute so the list can be filtered for partial or exact matches.

Adding or Replacing a Cemetery's Map after Creating it

There is currently no way to submit an image for a cemetery when editing it. This means the only chance to give a cemetery a map image is while creating it on the Add Cemetery Page. The solution is to add a map image field to the cemetery edit page.

Transform Position of Plots on Map Page

If a cemetery's plots are erroneously placed on the map, the only way to correct the mistake is to delete each plot and replace it with a new one, and add back any people who were evicted during deletion. Even for smaller cemeteries, this process would be extremely tedious and add an unnecessary risk to data integrity. The solution to this is to add a "Move Plots" button to the administrator Map Page that, when pressed, allows the administrator to click and drag one or more plots across the map and have the plot coordinates in the database update automatically.

Feedback on Add Person Page

Clicking the "Add Person" button on the Add Person Page does not give clear feedback on whether the submission was accepted or not. This can create uncertainty with successful submissions and confusion with unsuccessful submissions. The solution to this is to add distinct popups to appear when a submission is accepted or not, like those seen in the Add Cemetery Page.

Bugs

Overlapping Plot Icons on Map Page

A plot can be placed directly on top of another when adding plots to the cemetery map. This makes the plot underneath inaccessible without deleting the plot covering it. Possible solutions to prevent this bug include making it illegal for plots from the same cemetery to share coordinates or to define a radius around each plot in which new plots cannot be added. Alternatively, solutions to accommodate this bug include allowing the user to switch through the overlapping plots by double-clicking or right-clicking it, or consolidating the overlapping plots into a single icon that can be clicked to expand into a list of the plots it contains.

~~Deleting Plots via Map Page Interface Ineffective~~

~~Plots deleted via the map page interface will return upon refreshing the page. Additionally, if a plot is added after one is "deleted" via the map page, both the new plot and the old plot will have the same plot number upon refreshing the page. A possible solution to this bug is to have the map page use the same delete function that the plot list page uses.~~
Fixed as of 4/22/2025.

Assigning a Person to Multiple Plots

A person can be assigned to a plot even if they are already assigned to another. A possible solution for this bug is to add a boolean attribute to the Person table called “isBuried.” This attribute is set to false upon the person’s creation or when they are removed from a plot, and it is set to true when they are assigned to a plot. With this, the list of people who can be assigned to a plot is then limited to those with their isBuried attribute set to false.

Mapless Cemeteries on the Map Page

Selecting a cemetery with no map on the Map Page still shows the “Please select a cemetery to view the map” message in the map display. This ignores the fact that a cemetery has already been selected and does not explain that the cemetery has no map. The solution is to change the message in the map display to read “The selected cemetery has no map” if the selected cemetery has no map.